

Introduction

This article builds upon [the previous one](#), which focused on implementing a chatbot using the Pub/Sub pattern. Here, we'll concentrate on creating a multi-platform bot. A platform, in this context, refers to any communication channel, such as a messenger or a website widget. This is a sought-after solution: a single bot accessible across multiple platforms significantly increases the reach to potential users.

The main focus will be on developing an architecture that makes it easy to adapt the bot to new platforms. In this article, we will implement a chatbot working through Telegram and WebSocket with the potential for future expansion.

Practical Part

Architecture

The bot's architecture is based on modularity and scalability principles:

1. **Core Bot** — the central component responsible for message processing logic, completely independent of platforms.
2. **User Interaction Services:**
 - **Telegram Bot:** for communication via Telegram.
 - **WebSocket Chat:** for interaction via WebSocket.

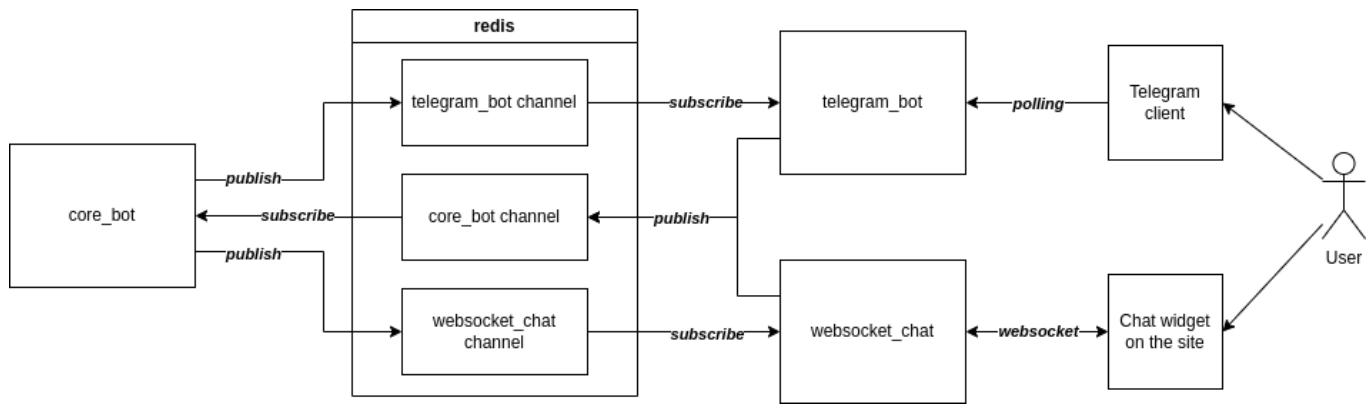
Redis Pub/Sub is used for inter-service communication. This solution ensures:

- Independence of services.
- Scalability (ability to distribute load).
- Simplicity in adding new platforms.

Workflow Overview

1. A user message is sent to the Core Bot through the appropriate Redis channel.
2. The Core Bot processes the message and sends the response back via the specified channel.

Redis provides bidirectional communication between services. The simplified interaction diagram looks like this:



System Deployment

To launch the components, `docker-compose` is used to describe four containers:

```

services:
  telegram_bot:
    build:
      context: .
      dockerfile: ci/telegram_bot/Dockerfile
    networks:
      - chat
    environment:
      - TELEGRAM_BOT_TOKEN={{ your-token }}

  websocket_chat:
    build:
      context: .
      dockerfile: ci/websocket_chat/Dockerfile
    networks:
      - chat
    ports:
      - "8000:8000"

  core_bot:
    build:
      context: .
      dockerfile: ci/core_bot/Dockerfile
    networks:
      - chat

  redis:
    image: redis:7.4
    networks:
      - chat
    ports:

```

```

    - "6379:6379"
    volumes:
      - redis-data:/data
    command: ["redis-server"]

networks:
  chat:
    driver: bridge

volumes:
  redis-data:
    external: true

```

The `Dockerfile` descriptions for all services are similar:

```

FROM python:3.12-slim

# Install dependencies
COPY ci/core_bot/requirements.txt .
RUN pip3 install --no-cache-dir -r requirements.txt

# Copy source code
WORKDIR /src
COPY src/core_bot .
COPY src/pub_sub.py ./pub_sub.py

ENTRYPOINT [ "python3", "run.py" ]

```

In this `Dockerfile`:

1. The official [Python image](#) is used as the base image.
2. Dependencies are installed using `pip`.
3. The source code is copied.
4. The entry point for the container is defined.

Implementation

After setting up deployment, let's proceed to implementation. The complete code is available in the repository: <https://github.com/anatoly-bobrovsky/multi-platform-chatbot>.

Dependencies

For each container, install the following dependencies:

1. **core_bot**:

```
redis==5.2.1 # pub/sub
```

2. **telegram_bot**:

```
aiogram==3.15.0 # telegram bot  
pydantic-settings==2.7.0 # for env  
redis==5.2.1 # pub/sub
```

3. **websocket_chat**:

```
fastapi==0.115.6 # API  
redis==5.2.1 # pub/sub  
uvicorn==0.33.0 # ASGI web server  
websockets==14.1 # websocket chats
```

Redis Pub/Sub

The main Redis logic is implemented in `src/pub_sub.py`, which includes:

- The `publish` function for sending messages to a specific channel.
- The `subscribe` function for handling incoming messages.

```
"""Redis Pub/Sub."""  
  
import json  
from dataclasses import asdict, dataclass  
from enum import Enum  
from typing import Awaitable, Callable  
  
import redis.asyncio as redis  
  
class Channel(Enum):  
    """Enumeration of available channels for Redis Pub/Sub."""  
  
    TELEGRAM_BOT = "telegram_bot"  
    CORE_BOT = "core_bot"  
    WEBSOCKET_CHAT = "websocket_chat"
```

```

@dataclass
class Message:
    """Data class representing a message to be sent through Redis Pub/Sub.

    Attributes:
        text (str): The text content of the message.
        chat_id (str): The identifier for the chat.
        callback_channel (Channel): The channel to which the message should
be sent.
    """

    text: str
    chat_id: str
    callback_channel: Channel

    def __str__(self) -> str:
        """Convert the message to a JSON string.

        Returns:
            str: JSON string representation of the message.
        """
        message_dict = asdict(self)
        message_dict["callback_channel"] = self.callback_channel.value
        return json.dumps(message_dict)

redis_client = redis.from_url("redis://redis:6379")

async def subscribe(channel: Channel, process_message: Callable[[Message],
Awaitable[None]]) -> None:
    """Subscribe to a channel and process incoming messages.

    Args:
        channel (Channel): The channel to subscribe to.
        process_message (Callable[[Message], Awaitable[None]]): The function
to process incoming messages.
    """
    async with redis_client.pubsub() as pubsub:
        await pubsub.subscribe(channel.value)

        while True:
            redis_message: dict = await
pubsub.get_message(ignore_subscribe_messages=True)
            if redis_message is not None:

```

```

        message_dict = json.loads(redis_message["data"])
        message_dict["callback_channel"] =
Channel(message_dict["callback_channel"])
        await process_message(Message(**message_dict))

async def publish(channel: Channel, message: Message) -> None:
    """Publish a message to a channel.

    Args:
        channel (Channel): The channel to publish the message to.
        message (Message): The message to be published.
    """
    await redis_client.publish(channel.value, str(message))

```

As it was indicated in the scheme before, there are 3 channels for each service:

TELEGRAM_BOT, CORE_BOT and WEBSOCKET_CHAT. The exchange will take place by messages, the logic of serialization of which is presented in the Message dataclass. After receiving the message, the service will process it at its own discretion (asynchronous function process_message). It should also be noted that Message has a separate attribute callback_channel, which is responsible for the service to which the response from core_bot should be returned.

WebSocket Chat

A FastAPI WebSocket server is implemented in src/websocket_chat/api_app.py. It processes incoming messages and sends them to the CORE_BOT channel, while responses from the WEBSOCKET_CHAT channel are sent back to the client.

```

@app.websocket("/chatbot/{chat_id}")
async def websocket_endpoint(websocket: WebSocket, chat_id: str) -> None:
    """
    WebSocket endpoint for chatbot communication.

    Args:
        websocket (WebSocket): The WebSocket connection object.
        chat_id (str): The unique identifier for the chat session.

    This function handles WebSocket connections for a chatbot. It accepts
    the WebSocket connection, then runs two asynchronous tasks concurrently:
    one for receiving messages from the client and another for sending
    messages to the client.
    """
    await websocket.accept()

```

```

await run_until_first_complete(
    (chat_ws_receiver, {"websocket": websocket, "chat_id": chat_id}),
    (chat_ws_sender, {"websocket": websocket, "chat_id": chat_id}),
)

```

The `chat_ws_receiver` and `chat_ws_sender` functions use the `publish` and `subscribe` functions defined above.

Telegram Bot

For Telegram, two asynchronous tasks are required: one subscribes to the `TELEGRAM_BOT` channel, and the other polls Telegram's API. The initialization is handled in `src/telegram_bot/run.py`:

```

async def main() -> None:
    """
    Main function to initialize and run the Telegram bot.

    This function sets up the bot with the token and default properties,
    defines a message processing function, and starts both the message
    subscription and the polling for updates.
    """
    bot = Bot(token=env.TELEGRAM_BOT_TOKEN,
default=DefaultBotProperties(parse_mode=ParseMode.HTML))

    async def process_message(message: Message) -> None:
        """
        Process a message by sending it to the appropriate chat.

        Args:
            message (Message): The message object containing chat_id and
text.
        """
        await bot.send_message(chat_id=int(message.chat_id),
text=message.text)

    await asyncio.gather(
        subscribe(channel=Channel.TELEGRAM_BOT,
process_message=process_message), dp.start_polling(bot)
    )

```

On the Telegram bot handlers themselves will be posting messages to the `CORE_BOT` channel:

```

@dp.message(CommandStart())
async def command_start_handler(message: TelegramMessage) -> None:
    """
    Handles the /start command.

    Args:
        message (TelegramMessage): The message object containing the
    command.
    """
    await message.answer("Hello!")

@dp.message()
async def message_handler(message: TelegramMessage) -> None:
    """
    Handles all incoming messages and publishes them to the core bot
    channel.

    Args:
        message (TelegramMessage): The message object containing the text
    and chat information.
    """
    await publish(channel=Channel.CORE_BOT,
message=Message(text=message.text, chat_id=str(message.chat.id),
callback_channel=Channel.TELEGRAM_BOT))

```

Core Bot

The Core Bot handles message processing. For example, an echo response is implemented in `src/core_bot/run.py`:

```

async def main() -> None:
    async def process_message(message: Message) -> None:
        await publish(channel=message.callback_channel,
message=get_echo_response(message))
    await subscribe(channel=Channel.CORE_BOT,
process_message=process_message)

```

But in your case it could be any complex processing logic you want, such as using a chain of LLM agents.

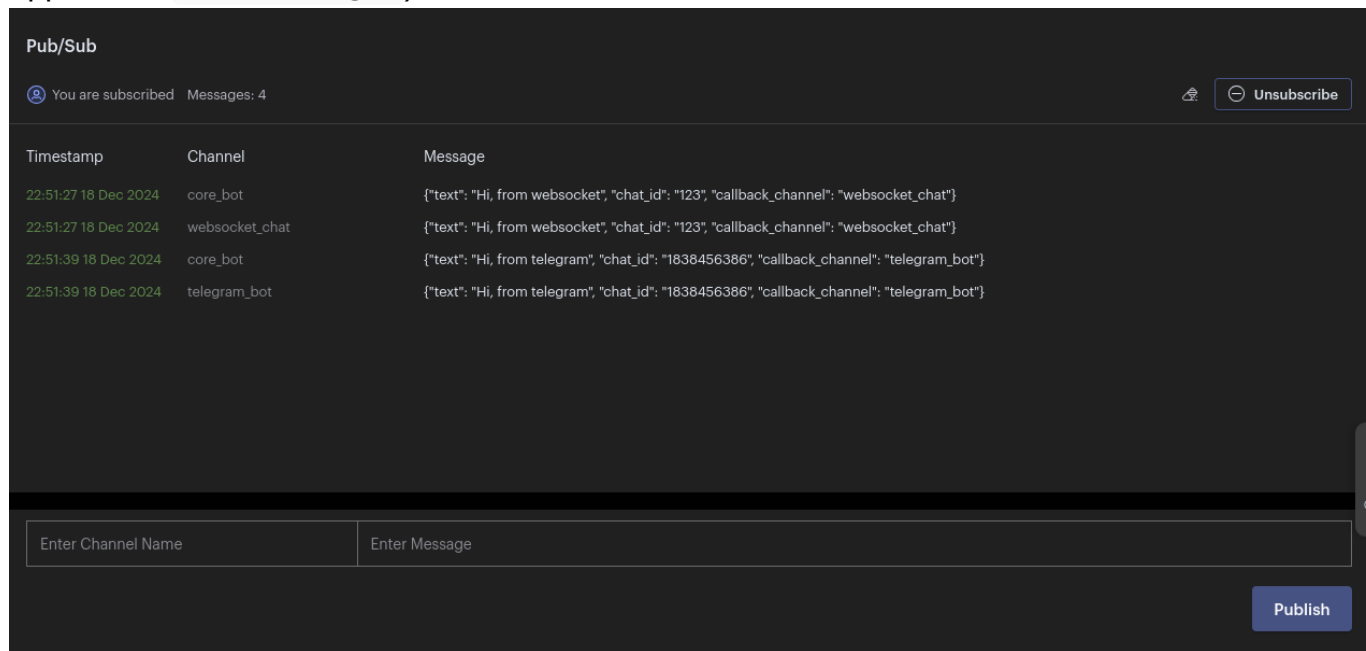
Running the Bot

To run the bot:

1. Create an external volume: `docker volume create redis-data`.
2. Set the `TELEGRAM_BOT_TOKEN` environment variable in `docker-compose.yml` to your bot's token.
3. Launch: `docker compose up -d --build`.

Websocket chat can be tested with `Postman` as in the last article, and Telegram bot can be tested with any Telegram client. In both cases, the bot will send the same message in response. Thus, one logic of interaction is transferred to several platforms!

To make sure that everything works correctly, let's look into `Redis` (as last time we will use GUI application `Redis Insight`):



Timestamp	Channel	Message
22:51:27 18 Dec 2024	core_bot	{"text": "Hi, from websocket", "chat_id": "123", "callback_channel": "websocket_chat"}
22:51:27 18 Dec 2024	websocket_chat	{"text": "Hi, from websocket", "chat_id": "123", "callback_channel": "websocket_chat"}
22:51:39 18 Dec 2024	core_bot	{"text": "Hi, from telegram", "chat_id": "1838456386", "callback_channel": "telegram_bot"}
22:51:39 18 Dec 2024	telegram_bot	{"text": "Hi, from telegram", "chat_id": "1838456386", "callback_channel": "telegram_bot"}

As you can see from the screenshot, messages are flying to the correct channels!

Conclusion

In this article, we developed and deployed a multi-platform chatbot using Python, Docker, and Redis Pub/Sub. This approach allows for easy functionality extension by adding new platforms — simply define a service to handle messages for your new platform and create a Redis channel for publishing messages!